

1. Project Overview and Objectives

This project is a compact but complete advertising analytics and conversion-prediction system. Its goal is to provide data insights, campaign KPI monitoring, and machine-learning–based conversion prediction in a single, easy-to-use interface. The system reads cleaned CSV datasets, trains a classifier on recent Google Ads performance, and then exposes both an interactive dashboard and a prediction tool through a web UI.

The backend is implemented with Flask, which is responsible for loading data, computing KPIs, aggregating statistics, and serving a prediction API. The frontend uses ECharts for interactive visualizations and a single `index.html` template that contains two logical views: a **Dashboard** tab and a **Predictor** tab. The trained model is serialized with `joblib` and loaded at runtime. Because the frontend never performs heavy computation, the UI remains responsive and the system is straightforward to deploy.

2. Data Processing and Training Set Construction

The training workflow is driven by a single cleaned CSV file (the most recent version you provided). The script reads this dataset and first normalizes the column names—for example, removing line breaks, trimming extra spaces, and standardizing names such as conversions, impressions, and `avg_cpc`. This ensures that later code does not depend on messy or OCR-style headers.

Next, the script harmonizes missing values by converting placeholders such as `--` or empty strings into `NaN`. This allows subsequent preprocessing steps (imputation and numeric casting) to be applied consistently. For the `ctr` column, if values appear as string percentages (for example, `"22.08%"`), they are parsed and converted into floating-point fractions (e.g., `0.2208`). This avoids typical issues such as `ValueError: could not convert string to float during model training`.

The training label converted is then constructed from the conversions column. The rule is simple and interpretable: if `conversions > 0`, the sample is labeled 1 (a conversion occurred); otherwise it is labeled 0. This yields a straightforward binary classification task: predicting whether a given impression pattern is likely to lead to at least one conversion, based on historical data.

3. Feature Engineering and Model Design

The model uses a focused set of five numerical features, all derived from the cleaned

training dataset and aligned with the dashboard metrics:

- clicks
- cost
- avg_cpc
- impressions
- ctr (stored as a float rather than a percentage string)

Since all selected predictors are numeric, the preprocessing pipeline remains relatively simple. The script applies a SimpleImputer with the "mean" strategy to fill any remaining missing values in these columns. This prevents failures due to NaN values while preserving the overall distribution of the data in a reasonable way.

The classifier is a RandomForestClassifier, configured with 300 trees, a maximum depth of 12, class_weight="balanced" to handle the slight imbalance between converted and non-converted samples, and a fixed random seed (random_state=42) for reproducibility. This design aims to balance expressive power with control over overfitting. After training, the model is saved as models/conversion_model.pkl using joblib.dump, which cleanly separates the offline training step from the online prediction service.

4. Dashboard View: KPI and Visualization Logic

The Dashboard tab (the default view in index.html) serves as an analytical cockpit. When a user visits the root route /, Flask loads the cleaned datasets, computes key metrics, and injects them into the Jinja2 template. All aggregations happen on the server side, so the frontend only receives ready-to-plot arrays and KPI values.

At the top of the page, a set of KPI cards displays core metrics such as total clicks, total impressions, average CPC, overall CTR, Google conversion rate, and total Google ad spend. These cards provide a quick summary of campaign health at a glance and reflect the most common performance questions a marketing analyst might ask.

Below the KPIs, the dashboard presents an ECharts line chart that shows the daily trend of clicks and cost over time, using shared dates on the x-axis and separate y-axes for clicks and cost. This helps highlight periods of unusual performance, rising budgets, or changes in efficiency. Alongside this, an hour-by-weekday heatmap reveals when the ads receive the most engagement: the x-axis is the hour of the day, the y-axis is the day of the week, and the color intensity represents click volume. This

visualization is particularly useful for discovering non-obvious “hot hours” and days where bidding or budgeting might need to be adjusted.

The second row of charts contains (1) a bar chart summarizing age-group performance (for example, clicks or conversions by age bucket) and (2) a pie chart comparing cross-channel spend (for example, Google vs. Facebook). These visualizations help answer questions such as “Which age group responds best to our ads?” and “How is our budget distributed across platforms?”. All charts are driven by numeric arrays computed in Flask and passed into the template via Jinja2 variables.

5. Predictor View: Online Conversion Probability Estimation

The Predict tab in the same `index.html` file provides a simple but powerful conversion-probability tool. The UI offers input fields for:

- Clicks
- Average CPC
- Impressions
- CTR
- Cost (optional)

If the user leaves **Cost** empty, the frontend automatically estimates it as $\text{clicks} \times \text{avg_cpc}$ before sending the request. This ensures that the model always receives a complete feature vector. For the CTR input, the JavaScript layer is designed to handle both fractional and percentage-like values: if the user enters a number greater than 1, it is interpreted as a percentage and converted to a fraction internally, keeping it aligned with the training-data representation.

When the user clicks the **Predict** button, the browser sends a JSON payload to the Flask endpoint `/api/predict`. The backend loads the saved `conversion_model.pkl` (if it is not already in memory), constructs a one-row `DataFrame` with the provided features, applies the same preprocessing pipeline as in training, and calls `model.predict_proba`. The response includes both the predicted probability and a binary label (1 for high conversion potential, 0 for low).

The frontend then displays the prediction in a user-friendly format: it shows the predicted conversion probability as a percentage and interprets the label as either “**High Conversion Potential**” or “**Low Conversion Potential** ⚠️”. This allows marketing teams to quickly test hypothetical scenarios (for example, “What if I

double my impressions but keep CPC similar?") and estimate how likely those setups are to produce conversions according to historical patterns.

6. Runtime Flow and Reproducibility

The runtime flow is split into two phases: **offline training** and **online serving**. In the first phase, the `train.py` script reads the cleaned CSV, normalizes column names, parses CTR, builds the converted label, trains the RandomForest model, and writes it to disk. This phase can be re-run whenever new historical data becomes available or the modeling strategy changes.

In the second phase, Flask acts as a lightweight web server. It loads the cleaned CSVs to compute KPIs and time series for the dashboard, and it loads the trained model for the prediction endpoint. Because the model is serialized with joblib, there is no need to retrain it at server startup, which keeps the app responsive and avoids long startup delays. On page load, the dashboard view renders directly from pre-computed values, so there are no empty charts or "undefined JSON" issues.

The architecture is friendly to local development on Windows with a standard Anaconda Python environment. Since all data and models are stored locally, there is no dependency on external GPUs or cloud services. To reproduce the full workflow, a user only needs to:

1. Ensure the cleaned CSV is in the expected data folder;
2. Run `train.py` once to produce `conversion_model.pkl`; and
3. Run the Flask app (`app.py`) and open a browser at `http://127.0.0.1:5000/`.

7. Key Files and Responsibilities

Although the UI is built from a single HTML template, the project is logically organized into a few core components:

- **train.py** – Reads the cleaned CSV, cleans and normalizes features (especially CTR), constructs the converted label, trains the RandomForest classifier, and saves the model as `conversion_model.pkl`.
- **app.py** – Flask backend that loads the cleaned data, computes KPI summaries, generates arrays for charts, loads the trained model, and exposes `/` (dashboard) and `/api/predict` (prediction endpoint).
- **templates/index.html** – Single HTML template containing both Dashboard and Predict views. Uses Jinja2 variables for KPIs and chart data and ECharts

for visualization.

- **static/style.css** – Provides the visual styling of the dashboard cards, layout, and overall look and feel.

Together, these components form a coherent system: data flows from historical CSVs into a trained model and aggregated KPIs, and then into an interactive dashboard and prediction interface that support decision-making for online advertising campaigns.